

Getting Started with Simulink CIGRE Build Process

This guide will help you set up and configure the CIGRE build process in Simulink. Follow these steps to get started.

Prerequisites

- Toolchain
 - Visual Studio for C compiler
 - 2017
 - 2019
 - 2022
 - MinGW
- MATLAB/Simulink ver. R2020a or later
 - Simulink Coder
 - Embedded Coder
- CIGRE mltbx

Toolchain Setup

Visual Studio

Install a supported version of Visual Studio, along with C/C++ toolchains.

MinGW

If not already set up, install MinGW support package through MATLAB:



You may need to restart MATLAB.

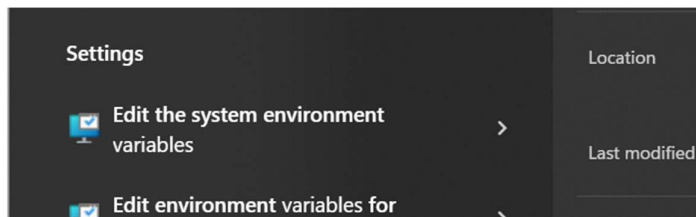
Run

```
compilers = mex.getCompilerConfigurations('C', 'Installed');  
minGW = compilers(ismember({compilers(:).Name}, 'MinGW64 Compiler  
(C)'))
```

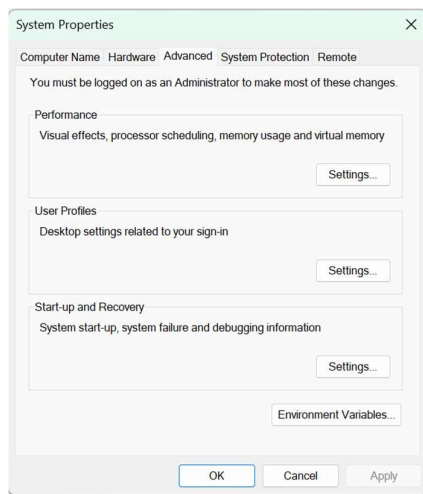
Add the path specified for the LinkerName to the path:

```
Details: [1x1 mex.CompilerConfigurationDetails]  
LinkerName: 'C:\ProgramData\MATLAB\SupportPackages\R2023b\3P.instrset\mingw_w64.instrset\bin\gcc'  
LinkerVersion: ''
```

Open System Environment Variables:



Go to “Advanced -> Environment Variables”



Add the path, e.g.

“C:\ProgramData\MATLAB\SupportPackages\R2023b\3P.instrset\mingw_w64.instrset\bin\” to the path”.

Restart MATLAB.

In MATLAB, run

```
!gcc
```

If you get the following error, it should be configured correctly:

```
>> !gcc  
gcc: fatal error: no input files  
compilation terminated.
```

```
>> |
```

Installation

- Install the CIGRE toolbox by running CIGRE.mltbx inside your MATLAB environment.
- Create the required CIGRE DLL toolchains by running:

```
cigre.install("VSVersion", <<version>>)
```

where <<version>> matches an installed version of Visual Studio.

This will create both 32- and 64-bit CIGRE DLL toolchains and register them with Simulink.

- Check the installation of the model advisor checks by:

```
cigre.test.advisorChecks
```

Model Configuration

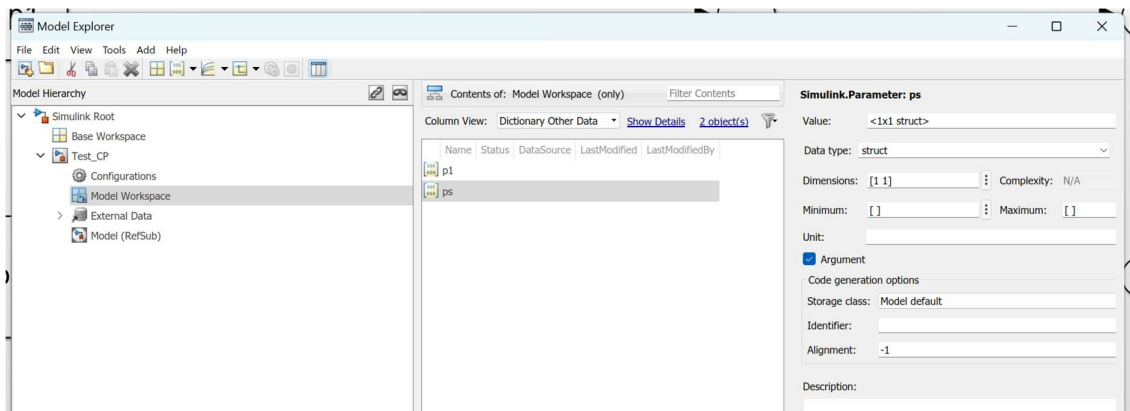
1. Open the *model configuration*.
2. Navigate to *Hardware Target* and choose whether you want to generate a 32- or 64-bit DLL.
3. Navigate to *Code Generation* and set the *System target file* to 'cigre.tlc'
4. Select the appropriate CIGRE DLL toolchain

Model Parameters

To support true multi-instance parameters, i.e. so the DLL supports parallel execution, we must avoid generated C code where the parameters that are stored in the global namespace. Any parameter that is intended to be tuneable should be:

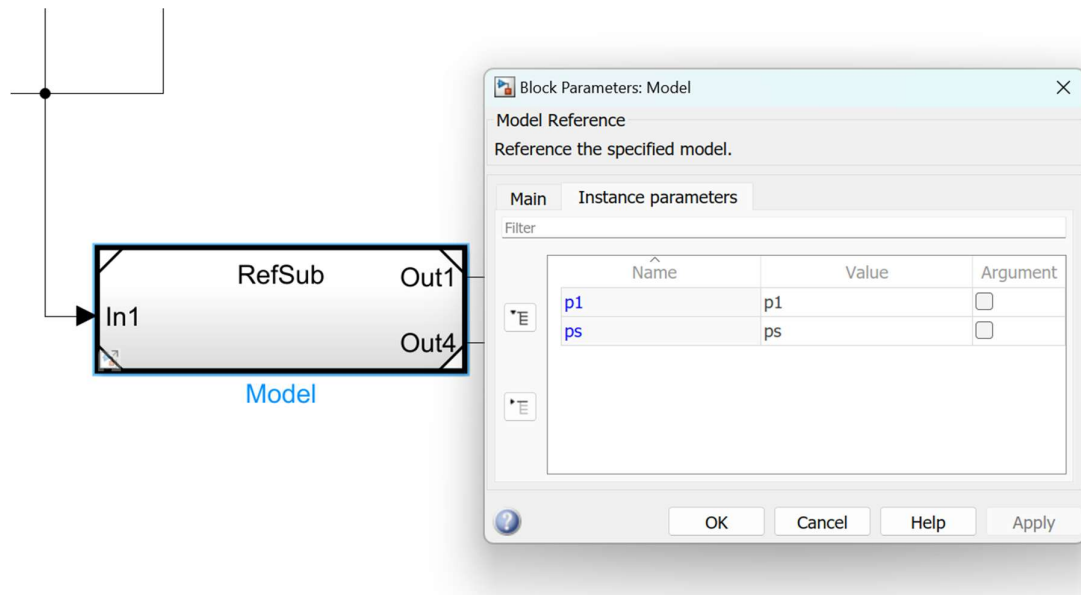
- a Simulink.Parameter object
- defined as a model argument in the top level model
- stored in the model workspace of the top level model
- have the storage class set to "model default"

For example, the screenshot below shows a Simulink.Parameter, ps, of type struct:

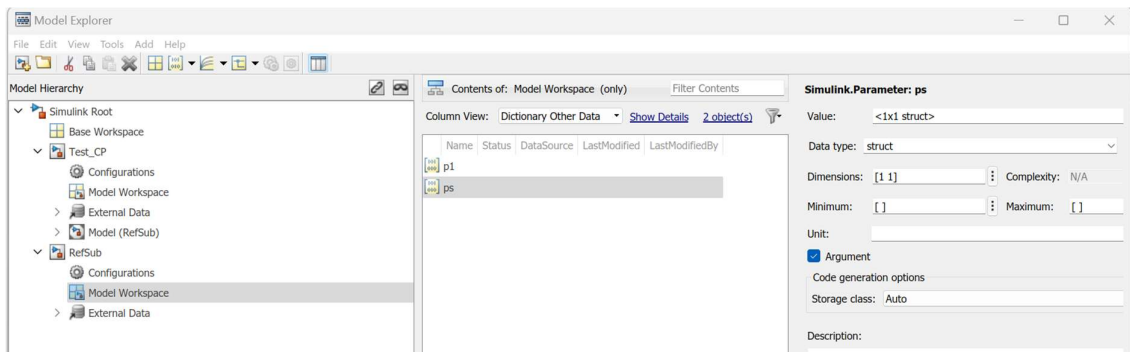


In this example, the parameter is a structure of parameters. Each of the “leaves” of the struct will be accessible as individual parameters in the CIGRE dll. In Simulink, you refer to the parameter as e.g. *ps.field1.otherField*.

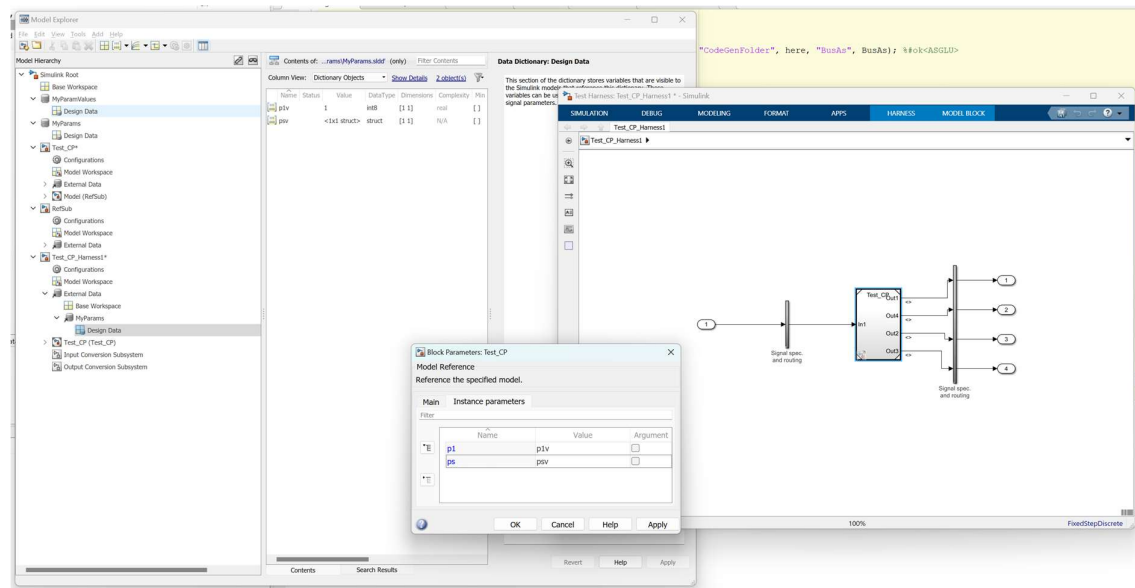
If a parameter is used in one or more model references, this parameter should be passed to down the reference hierarchy using model arguments, e.g.



In the model reference, the parameter is again defined in the model workspace. This time, the storage class can be set to Auto e.g.,



If you wish to store the parameter value in a data dictionary, this value should be passed into the top model using e.g. a test harness:



Model Checker

To check if a model is compatible with the CIGRE DLL code generation run checks on the model.

```
cigre.checkModel(<<modelName>>)
```

```
>> cigre.checkModel('interfacetypes_0003_fail')
Running Model Advisor...

Systems passed: 0 of 1

Systems with information: 0 of 1

Systems with warnings: 0 of 1

Systems with failures: 1 of 1

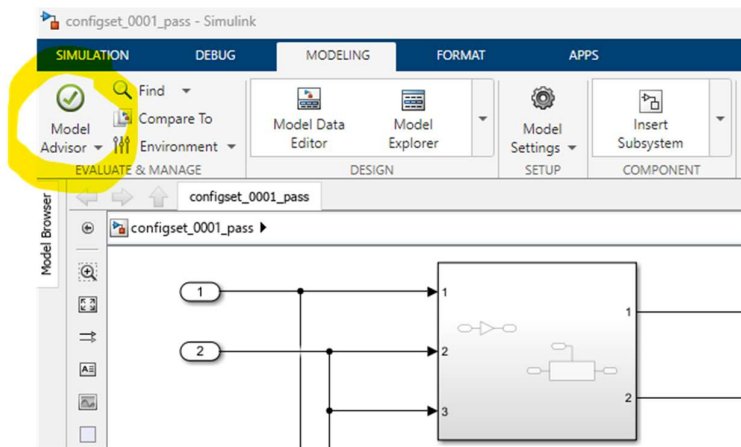
Systems with justifications: 0 of 1
Summary Report

====> ***** Model failed some checks see report for details!

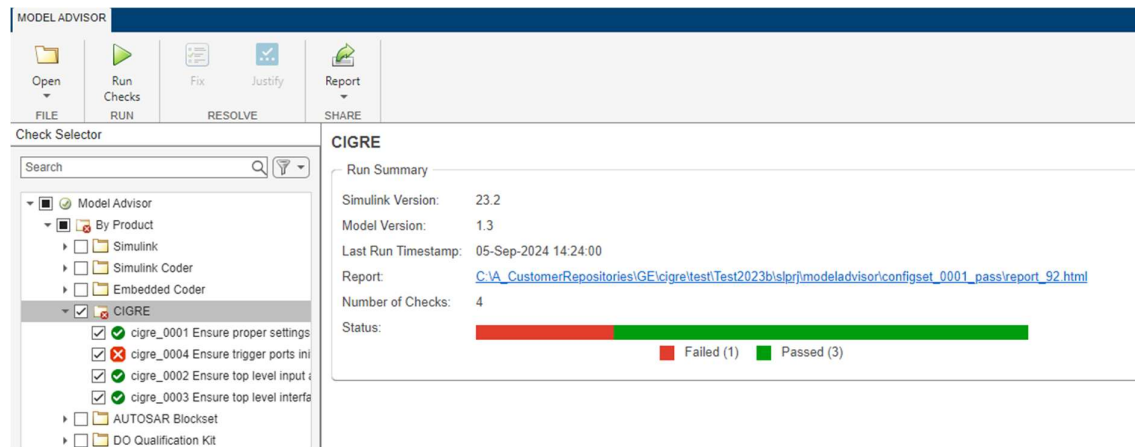
>> |
```

For details press the Summary Report link.

To run the model advisor interactively, open the model advisor from the modelling pane.



Select the CIGRE checks under the By Product folder:



And run them.

Building the Model

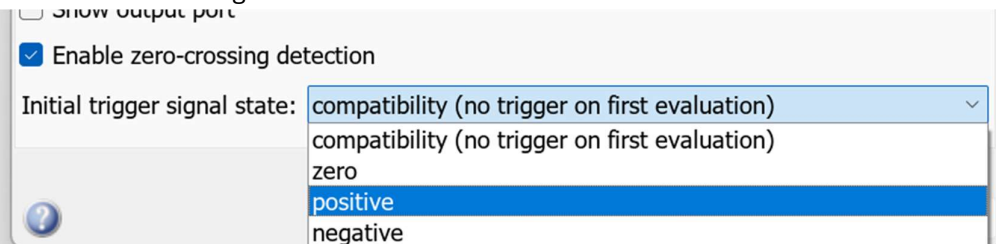
1. The model is build by calling

```
cigre.buildDLL(<<modelName>>)
```

where <<modelName>> is the name of the top level model that you wish to build. This will create a DLL called <<modelName>>_CIGRE.dll, along with a header <<modelName>>_CIGRE.h that describes the DLL interface.

Known Limitation

- The top-level model cannot contain virtual buses.
- The top-level model must support multiple instances.
- Logical/Boolean data in Simulink is converted to int32 in CIGRE.
- Compatibility mode for triggered subsystem initialisation is not supported due to how the initialisation code is generated:



- (u)int64 is not generally supported as this requires support for multi-word typedefs.
- Must use fixed time-step.
- Model must have at least one input.

PSCAD Import

When importing the model into PSCAD a lot of users use the import tool provided by Electronix: [Request A Copy](#) (19/09/2025).

In this tool, you should set the “Time to release initial conditions” to 0, rather than using the default value of 1. You should also make sure the Memory Snapshotting checkbox is ***not*** checked. Memory snapshotting is dealt with by the DLL itself.

